

Gray Interface '26

Task 4

1. INTRODUCTION & OBJECTIVE

In this assignment, you will build an end-to-end **tree-based ensemble learning pipeline** using a real Kaggle competition. You will explore **Decision Trees, Random Forests, Boosting, SHAP, and model ensembling**, along with EDA, feature engineering, hyperparameter tuning, and deployment.

2. THE ASSIGNMENT: BUILD & EVALUATE MODELS

Dataset:

<https://www.kaggle.com/competitions/playground-series-s6e6/data>

Perform appropriate preprocessing before training your models.

Your tasks include:

- Perform EDA and analyze the class distribution.
- Analyze numerical features using appropriate visualizations.
- Check for missing values, duplicates, outliers, and correlations.
- Perform appropriate preprocessing and feature engineering.
- Analyze feature importance and identify potentially redundant features.
- Use **Stratified K-Fold Cross-Validation**.

Decision Tree & Random Forest

- Train a **Decision Tree Classifier**.
- Experiment with `criterion`, `max_depth`, `min_samples_split`, and `min_samples_leaf`.
- Visualize the Decision Tree.
- Train a **Random Forest Classifier**.
- Experiment with `n_estimators`, `max_depth`, `max_features`, and `min_samples_leaf`.
- Compare Decision Tree and Random Forest.

Boosting & Ensemble Learning

- Train **AdaBoost**, **Gradient Boosting**, and **XGBoost**.
- **Optional**: LightGBM/CatBoost.
- Experiment with `n_estimators`, `learning_rate`, `max_depth`, and other relevant hyperparameters.
- Compare the performance of all models.
- Implement **Blending** or **Stacking** using your best-performing models.
- Compare the ensemble against the best individual model.

SHAP Analysis

- Apply SHAP to the best-performing tree-based model.
- Generate a SHAP feature importance plot and summary/beeswarm plot.
- Generate dependence plots for **2 important features**.
- Explain **2 individual predictions** using SHAP.

Evaluation & Kaggle

- Evaluate using **Balanced Accuracy**, **Accuracy**, **Precision**, **Recall**, **F1-Score**, and **Confusion Matrix**.
- Generate the Kaggle submission using the `test.csv`, submit it, and record your leaderboard score and attach it in the README.
- Compare your cross-validation performance with your Kaggle leaderboard performance.

Web Application

- Save the best-performing model/ensemble.
- Build a simple web interface using **Flask/FastAPI** for your final model.
- Allow users to enter the required features and display the predicted stellar class (**Galaxy / Star / Quasar**).
- **Optional**: Deploy the application using a free hosting platform.

3. DELIVERABLES

Submit a public link to your **Google Colab/Kaggle Notebook(s)** along with a **README.md** file. Ensure that all viewing permissions are set to public.

- Submit **README.md** describing your workflow, preprocessing, feature engineering, model training, tuning, evaluation, SHAP analysis, and conclusions.
- Include the Kaggle submission leaderboard score in the README.
- Include a **public Google Drive link to a short demo video** of your working web application in the README.md.
- The documentation **does not need to be lengthy**, but it **must be written in your own words** and should clearly explain your understanding of the work performed.
- Upload all notebook files and the README.md to the GitHub repository under your own folder.
- **Optional:** Deploy the application using a free hosting platform like Vercel/Render and include the live deployment link in the README.md.

Good luck! Remember: Don't just chase the leaderboard score. Explain **what you changed, why you changed it, and how it affected model performance.**